ESTR1002 Fall 2025

Lab Exercise #10

Due: 3 Dec 2025

Separate your answer for each (sub)question in a different .c file, except for the last question (that consists of multiple files). Compress all files into a .zip and submit it to Blackboard.

The lab this time tests your understanding of pointers and arrays. Once you can totally understand your own (workable) solution, you have mastered this topic.

Question 1

In fact, the main function can have up to 3 parameters. For example

```
int main(int argc, char **argv, char **envp){
    // your codes
}
```

The first two parameters are for the arguments when you execute your program. The third one is for the “environment variables”.

The environment variables include the location of your home directory, terminal settings, login name, etc. This can provide some information about your environment to your program.

Actually, `envp` points to an array. The array stores the addresses of the strings. Each string corresponds to one environment variable.

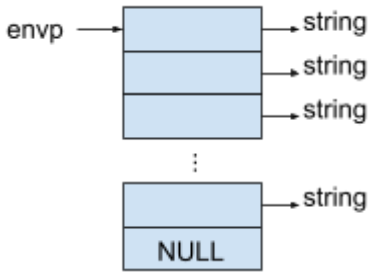
What are strings in C? **A string in C is a char array.** The null character '\0' indicates the end of a string. For example the string "Hi" is a char array of at least 3 entries, which stored 'H', 'i', and '\0'.

When you want to print a string, you only need to know the address of the first character in the string. For example if you want to print the first string in `envp`, you can use the code

```
puts(envp[0]);
```

or equivalently

```
printf("%s\n", envp[0]);
```



We do not know how many items are in the array, but we know the last item is NULL. The figure shows the relation between `envp` and the strings.

As we have covered pointers in the lecture, let’s practice!

Task 1: Print out all the strings in the figure without using array notations. i.e. DO NOT use something like `envp[i]` in your code.

Note 1: Of course you can first do a version using array notations for checking the output.
Note 2: You can write `char *envp[]` instead of `char **envp` in the main declaration.
Note 3: You can also print the environment variables in your terminal.

1. If you are using a Mac / Linux / Unix, run the terminal and input the command `env`
2. If you are using Windows, run cmd and input the command `set`

There are many lines in the output of Task 1. If you look at it carefully, you can find your login name (username) in one of the lines! Specifically, it is only a part of the line.

Task 2: In some applications, you may need to extract information from the environment variables. Now let's extract your login name (username).

Question 2

Your tutor wants to find out who is the most hardworking student in class! He wants to input the names according to the order they fell asleep, then print their names in the reverse order. Although you can create an array large enough to store all names, your tutor doesn't want to waste our precious memory!

The first line of the input is an integer x, which indicates the number of students in class. Then there are x lines of strings. Each line corresponds to a student's name. Assume a name is at most 14 characters.

Task 1: Read the students' names from user input and pass the data to your tutor's function. You are requested to create an array with x strings. Your tutor has given you the skeleton code. Please complete the main function to achieve your task.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void foo(char **names, int x)
{
    int i;
    for (i = x-1; i >= 0; i--)
        printf("%s\n", names[i]);
}

int main(void)
{
    // write your codes here
    foo(names, x);
    // write your codes here
    return 0;
}
```

Sample Input:	Sample Output:
3 George Bush Bill Gates Steve Jobs	Steve Jobs Bill Gates George Bush

Your tutor suddenly realized that you have learned struct already! So he says: “Let’s change the code and use structure!”

Task 2: Your task is the same, but your tutor has given you another (terrible) skeleton code below. Please complete the main function to achieve your task.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int no;
    char (*names)[16];
} myList;

void foo(myList *bar)
{
    int i;
    for (i = bar->no-1; i >= 0; i--)
        printf(“%s\n”, bar->names[i]);
}

int main(void)
{
    // write your codes here
    foo(bar);
    // write your codes here
    return 0;
}
```

Your (annoying) tutor says: “I don’t like the design of the structure ._.”

Task 3: You are given another structure declaration below. Complete the same task as in Task 2.

```
typedef struct {
    int no;
    char names[][16];
} myList;
```

Question 3

Once upon a time, there was a structure… The structure contained a huge, huge integer array. The king wrote some random numbers in the array and wanted you to write a function which can find out the maximum number in the array.

The king assigned you this honour task. You were requested to complete the code in three files: main.c, func.c and func.h

You can compile the files into an executable “lab10q3” by the command:

```
gcc main.c func.c -o lab10q3
```

The code in “func.h” is simple

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 100000000 // 100,000,000

struct foo {
    int a[SIZE];
};
```

Here is the code in “main.c”

```
#include "func.h"

static struct foo bar;

// write your function prototype here

int main(void)
{
    int i;
    srand(time(NULL)); // random seed
    for (i = 0; i < SIZE; i++)
        bar.a[i] = rand(); // fill in random numbers
    // call your function here
    return 0;
}
```

Here is the code in “func.c”

```
#include "func.h"

// write your function body here and print out the max number
```